Models and methods used in this project

This document describes every statistical and machine-learning model in the pipeline: what it is, why it's here, where it runs, the exact settings used, and its limitations. Models are grouped by the role they play. All randomness is seeded (SEED = 42, src/config.py).

---

0. Evaluation protocol (shared by everything)

- **Patient-level, stratified 7:3 split** (01_feature_selection.py::patient_split). Patients are shuffled within each histology and assigned whole to train or test, so no individual contributes samples to both sides. The split is created once, saved to data/processed/split.json, and reused by every selector and classifier. That keeps the method comparison fair, as the paper also required.
- **Hyperparameters are tuned only inside the training set**, with 5-fold cross-validation. The test set is scored once per model.
- **Uncertainty**: 10,000-replicate bootstrap of the test predictions gives a 95% CI for accuracy (utils.bootstrap_ci).
- **Binary convention**: LUAD = positive and LUSC = negative, as in the paper. Sensitivity, specificity, precision and F-measure are computed from the confusion matrix (utils.binary_stats).

---

1. Preprocessing model

TMM normalization + log2-CPM

- **What:** edgeR's Trimmed Mean of M-values. Each sample gets a scaling factor from the trimmed mean of its log fold-changes against a reference sample. That corrects for library size and for RNA composition, e.g. a few extremely highly expressed genes soaking up reads. Counts are then converted to counts-per-million and log2-transformed (prior count 1).
- **Why:** it removes the mean–variance dependence of raw counts, so genes are comparable across samples. TPM is designed for within-sample comparisons and is the wrong choice for cross-sample ML.
- **Where:** utils.tmm_logcpm (via conorm.tmm), called in 00_preprocess.py.
- **Filter:** genes are dropped if CPM < 1 in more than 600 tumour samples (the paper's rule), leaving 15,686 genes.

---

2. Feature-selection models (the five "voters")

Each method runs on the training split only and returns a gene set. The overlap of those sets is the paper's central idea.

2.1 Differential gene expression (DGE)

- **What:** a per-gene two-sample **Welch t-test** on log-CPM between LUSC and LUAD. P-values are corrected with **Benjamini–Hochberg FDR**, and logFC = mean(LUSC) − mean(LUAD).
- **Selection:** keep genes with FDR < 0.05 and |logFC| > 0.5, then take the **top 500 by p-value**.
- **Why:** the classical univariate test for biomarkers. It is easy to interpret, but it ignores correlation between genes and is sensitive to outliers.
- **Deviation:** the paper used edgeR's negative-binomial model. Bioconductor was unavailable in the run environment, so the Welch test on normalized log-CPM stands in for it. Its rankings are highly concordant for strong signals like these.
- **Where:** 01_feature_selection.py::run_dge / select_dge.

2.2 Principal Component Analysis (PCA)

- **What:** an unsupervised linear projection onto orthogonal axes of maximal variance.
- **Selection:** genes are z-scored and PCA is fit with 10 components. The **top 500 genes by |loading| on PC1** are kept.
- **Why:** it is label-free, so it gives an independent view of which genes drive the dominant structure. In TCGA lung tumours, PC1 is largely the LUAD/LUSC axis.
- **Limitation:** the result depends on the scaling choice. The sensitivity analysis also tests centring without scaling.
- **Where:** select_pca, plus 06_sensitivity.py::pca_centered.

2.3 Minimum-Redundancy Maximum-Relevance (mRMR)

- **What:** greedy filter selection. At each step it adds the gene that maximizes **relevance − mean redundancy**:
  - relevance: ANOVA F-statistic against the label, min-max scaled to [0, 1]
  - redundancy: mean |Pearson r| with the genes already chosen

  This is the "MID" scheme.
- **Selection:** 500 genes, searched over the **full** gene set.
- **Implementation note:** this is a vectorized NumPy version. Each step is one matrix-vector product on a z-scored matrix, so the full run takes about 1 second. The off-the-shelf package was too slow, and it collapsed onto the DGE ranking: 469 of 500 genes were identical. That happened because unscaled F-statistics (hundreds) swamp a redundancy term bounded by 1. Scaling relevance fixed it, and the DGE overlap dropped to about 220 genes.
- **Why:** it deliberately avoids picking many near-duplicate correlated genes, which complements DGE.
- **Where:** select_mrmr.

2.4 XGBoost (gradient-boosted decision trees)

- **What:** an additive ensemble of shallow regression trees fit sequentially to the gradient of the log-loss, with row and column subsampling. It captures non-linear effects and interactions.

- **Settings:** 400 trees, max_depth 4, learning rate 0.1, subsample 0.8, colsample_bytree 0.5.
- **Selection:** genes ranked by gain importance, keeping the top 148 (the paper's XGBoost set size).
- **Why:** it is a strong non-linear learner, and it is robust to outliers and monotone transforms.
- **Limitation:** it can overfit, and gain importance spreads across correlated features.
- **Where:** select_xgb.

2.5 L1-regularized logistic regression (LASSO)

- **What:** logistic regression with an L1 penalty, which drives most coefficients exactly to zero and so performs embedded feature selection. Solved with liblinear on z-scored genes.
- **Tuning:** 20 values of $C = 1/\lambda$, log-spaced from 1e-3 to 10, with stratified 5-fold CV. The **1-SE rule** picks the most regularized model whose CV accuracy is within one standard error of the best, as glmnet does. This yields 18 genes.
- **Why:** it gives a sparse linear model that is interpretable and less prone to overfitting.
- **Limitation:** it is unstable among correlated genes and tends to pick one per cluster. The accuracy-optimal λ kept 1,588 genes, which is why the 1-SE rule matters.
- **Where:** select_lasso. The sensitivity analysis also searches for the λ giving about 68 genes, the paper's size (06_sensitivity.py::lasso_target).

2.6 The overlap ("ensemble vote")

- Genes chosen by **≥2 methods → 339-gene panel**, used for classification and pathways.
- Genes chosen by **≥3 methods → 140 candidate biomarkers**.
- Genes chosen by **all 5 → 6 genes**: BCL2L15, DDAH1, DSC3, DSG3, KRT5, SIAH2.
- The overlap is where the ensemble does its work: it keeps genes that pass several filters with different failure modes.

---

3. Classification models

3.1 Random Forest (main validator)

- **What:** a bagged ensemble of decision trees. Each tree is grown on a bootstrap sample, with a random subset of features considered at each split. Predictions are majority votes and class probabilities are vote fractions.
- **Role:** it validates **every** gene set on the same split, so the comparison between selectors is like-for-like. It stands in for R's ranger, which the paper used.
- **Tuning:** GridSearchCV, 5-fold on the training set, scored by accuracy, over:
  - n_estimators ∈ {300, 500}
  - max_features ∈ {$\sqrt{p}$, $0.1 \cdot p$}
  - min_samples_leaf ∈ {1, 3}
- **Where:** 02_classification.py::fit_eval.

- **Result:** every gene set scores 0.958–0.968 accuracy and 0.982–0.991 AUROC. The ≥2 overlap panel is the best (0.968 / F 0.970).

3.2 Random Forest, three-class (LUAD / LUSC / Normal)

- **What:** the same algorithm with 500 trees and multi-class output. It is scored by accuracy, macro-F1 and one-vs-rest macro-AUROC.
- **Where:** 05a_three_class.py.
- **Result:** the 339-gene panel reaches macro-F1 0.957. All 15,686 genes reach 0.964.

3.3 Single-gene ROC classifiers (biomarker evaluation)

- **What:** each gene's expression is used on its own as a continuous score. AUC is the probability that a random LUAD sample scores higher than a random LUSC sample. It is oriented so AUC ≥ 0.5.
- **Why:** this reproduces the paper's biomarker evaluation. It answers how useful a gene is alone.
- **Where:** 03_biomarkers_roc.py, on TCGA and on the external GSE28582 microarray cohort (probes collapsed to genes by maximum).

3.4 XGBoost + Random Forest for the feature-count ablation

- **What:** an XGBoost model on all genes ranks features by gain. Random forests of 400 trees are then retrained on the top-k genes, for k = 2 … 500, and scored on the test set.
- **Where:** 05b_shap_ablation.py.
- **Result:** 2 genes give AUROC 0.955, 10 give 0.980, and 17 give 0.984. The curve plateaus around 25 genes.

---

4. Interpretability

SHAP (SHapley Additive exPlanations), TreeSHAP

- **What:** it splits each prediction into additive per-gene contributions, using Shapley values from cooperative game theory. For tree ensembles these are computed exactly.
- **Model explained:** XGBoost on the 339-gene panel (400 trees, depth 4, learning rate 0.08).
- **Implementation:** the SHAP values come from XGBoost's native pred_contribs=True, which is exact TreeSHAP. The shap package is used only for plotting. Its TreeExplainer can't parse the xgboost 3.x model format.
- **Output:** a beeswarm and a mean-|SHAP| ranking. The top drivers include DSC3 and KRT5, both canonical squamous markers.

---

5. Survival models

5.1 Kaplan–Meier estimator + log-rank test

- **What:** a non-parametric estimate of the survival function S(t) under right-censoring. Within each histology, patients are split at the gene's median expression, and a log-rank test compares the two curves.
- **Why:** it replaces the paper's KM-Plotter step with TCGA's own clinical data.

5.2 Cox proportional-hazards regression

- **What:** a semi-parametric model with $h(t|x) = h_0(t) \cdot \exp(\beta x)$. Each gene is fit univariately as a continuous covariate, giving a hazard ratio (HR) $\exp(\beta)$ per log2-CPM unit.
- **Where:** 05c_survival.py, using lifelines.
- **Result:** 63 of 280 gene–histology tests have log-rank $p < 0.05$. KRT6A and KRT17 are unfavourable in LUAD, as the paper reports.
- **Caveats:** these are univariate tests, without adjustment for stage, age or treatment, and without multiple-testing correction. Treat them as exploratory.

---

6. Unsupervised and robustness models

6.1 Stability selection

- **What:** the training set is bootstrapped 30 times. On each resample, a fixed-penalty L1 logistic regression ($C = 0.1$) and a small XGBoost model (150 trees, depth 3) are fit, and the union of their selected genes is recorded. The selection frequency across resamples measures how reproducibly a gene is chosen. To keep this tractable, it runs over the top 2,000 genes by F-statistic.
- **Why:** a single run of any selector is noisy. Stability identifies genes that keep being chosen, the idea behind Meinshausen & Bühlmann's stability selection.
- **Where:** 05d_novel.py::stability.
- **Result:** stable, high-AUC candidates that the paper doesn't highlight: DSG3, CERS3, SERPINB13, BNC1, MIR205HG, DDAH1.

6.2 Agglomerative hierarchical clustering + silhouette

- **What:** bottom-up clustering (Ward linkage) of tumours within each histology, on the z-scored 339-gene panel, for $k = 2, 3, 4$. The silhouette score measures cluster separation.
- **Result:** LUSC shows moderate structure (silhouette 0.30 at $k = 2$). LUAD shows little. This hints at LUSC molecular subtypes.

6.3 Confound audit (Mann–Whitney U)

- **What:** for each biomarker, a rank-sum test of male vs female **within** each histology, with BH-FDR correction. Testing within histology matters: LUSC skews male, so a pooled test would just recover the LUAD/LUSC difference.

- **Result:** the key discriminators KRT5, KRT6A, NECTIN1 and PERP are not sex-associated.

6.4 Sensitivity analysis of the overlap

- **What:** the overlap is recomputed under alternative selector settings, and a fixed 500-tree random forest is scored on each panel. The settings are:
  - paper-matched set sizes
  - PCA centred without scaling
  - varying the top-N taken from DGE, PCA and mRMR (50–500)
- **Where:** 06_sensitivity.py.
- **Result:** the overlap size is driven by top-N. N ≈ 150–175 gives the paper's ≈131 (≥2), and N = 50 gives exactly 17 (≥3). Accuracy stays at 0.961–0.968 across every setting.

---

7. Statistical tests used for pathways

- **Over-representation analysis** uses a **hypergeometric / Fisher's exact test** of each GO, KEGG and Reactome gene set against the ~15.7k analysed genes as background, with BH adjustment (gseapy enrich, Enrichr libraries).
- It is run separately for genes up in LUSC and genes up in LUAD (04_pathways.py). The top LUSC term is keratinization.

---

8. Leakage audit (read this before trusting any number)

Being explicit about where information could leak is part of doing this properly:

Step	Status
Train/test split	Patient-level and stratified; the test set is never used for tuning. (OK)
Feature selectors (DGE, PCA, mRMR, XGBoost, LASSO)	Fit on training samples only. (OK)
Hyperparameter tuning	CV inside the training set only. (OK)
Low-count gene filter and TMM normalization	Computed on all tumour samples, including test. Caveat: Mild: both are unsupervised (no labels) and per-gene/per-sample. A stricter pipeline would fit them on train and apply them to test.
Single-gene biomarker AUCs (TCGA)	Computed on the full cohort, like the paper. The genes were chosen on the training set, so these AUCs are slightly optimistic. Caveat: The external GSE28582 validation (mean AUC 0.925) is the unbiased check.
Three-class task	Uses the 339 genes chosen on the binary split, but a different split that includes

	normals. Some three-class test tumours were in the feature-selection training set, so the overlap panel's three-class score is slightly optimistic. Caveat: The all-genes baseline (macro-F1 0.964) is unaffected.
Survival	Univariate and unadjusted, with no multiple-testing correction. Caveat: Exploratory only.

None of these caveats change the conclusions: every classifier is ≥ 0.95, and the biomarkers hold up on an independent platform. They are the obvious next hardening steps if this became a production analysis.